

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

INGENIERÍA EN SISTEMAS



Glosario de Términos

BioMove App v4.0

Análisis biomecánico de sentadilla con visión artificial e IA personal

Versión:	2.0 — Final
Fecha:	13 de junio de 2026
Autor:	Bryan Xavier Calderón Novillo
Total de términos:	62 términos — orden alfabético

Introducción

El presente glosario define los 62 términos técnicos, biomecánicos, estadísticos y tecnológicos fundamentales del proyecto BioMove App v4.0. Está organizado en orden alfabético y cada término incluye: definición precisa en el contexto del proyecto, categoría temática y fuente bibliográfica o de implementación. El glosario está diseñado para facilitar la comprensión de la tesis a lectores con distintos niveles de especialización: biomecánicos, ingenieros de software y evaluadores académicos.

— A —

Acl_risk_score (Score de riesgo ACL) <i>Biomecánica / IA</i>	Indicador de riesgo de lesión del ligamento cruzado anterior (LCA), expresado en escala 0-100. Se calcula en <code>squat_analyzer_40.py</code> a partir del ángulo de valgo de rodilla, velocidad angular y profundidad de sentadilla. Valores >50 se consideran de riesgo significativo. Fuente: <i>squat_analyzer_40.py — BioMove v4.0; Silva et al. (2019). Knee valgus biomechanics. J Biomech.</i>
Alpha de Cronbach (α) <i>Estadística / ISO 25010</i>	Coeficiente de consistencia interna del instrumento de medición. Valora el grado en que los ítems del cuestionario miden el mismo constructo. Valor obtenido en BioMove: $\alpha=0.918$ (escala: excelente si $\alpha>0.90$). Calculado con IBM SPSS Statistics v29 sobre los 15 ítems del cuestionario híbrido ISO+SUS+UMUX. Fuente: <i>George, D. & Mallery, P. (2003). SPSS for Windows step by step. Pearson. / ISO/IEC 25023:2016.</i>
Annotated_download_url <i>Backend / Base de datos</i>	Campo VARCHAR(500) en la tabla <code>video_jobs</code> que almacena la URL RELATIVA del video anotado generado por <code>video_annotator.py</code> . El frontend Flutter debe concatenar <code>AppConstants.baseUrl</code> antes de esta URL para que <code>VideoPlayer</code> pueda reproducirlo. Ejemplo: "/exports/annotated/{job_id}_annotated.mp4". Fuente: <i>video_jobs (SQLAlchemy) — BioMove v4.0; GoRouter / Dio — Flutter.</i>
API REST (Application Programming Interface) <i>Arquitectura de software</i>	Interfaz de programación de aplicaciones que sigue los principios de la arquitectura REST (Representational State Transfer). BioMove utiliza FastAPI para exponer 15+ endpoints REST (POST <code>/videos/upload-and-analyze</code> , GET <code>/videos/{job_id}/status</code> , GET <code>/best-reps/{exercise}</code> , etc.). La comunicación entre Flutter y FastAPI se realiza mediante HTTP/HTTPS con autenticación JWT.

	Fuente: <i>Fielding, R.T. (2000). Architectural Styles and the Design of Network-based Software Architectures. UCI. / FastAPI 2024.</i>
--	--

aiosqlite <i>Backend / Base de datos</i>	Adaptador asíncrono para SQLite que permite usar SQLite con Python async/await. Utilizado en BioMove para el motor de base de datos en desarrollo: "sqlite+aiosqlite:///./biomove.db". Elimina el bloqueo del event loop de FastAPI durante las operaciones de base de datos.
	Fuente: <i>aiosqlite 0.19.x (PyPI). / SQLAlchemy 2.x Documentation.</i>

— B —

Bar_velocity_ms (velocidad de barra) <i>Biomecánica / VBT</i>	Parámetro biomecánico calculado en <code>squat_analyzer_40.py</code> . Estima la velocidad media de la barra en metros por segundo durante la fase concéntrica, inferida del desplazamiento vertical de los keypoints de cadera de MediaPipe. Clave para el VBT (Velocity Based Training) y el cálculo de 1RM del día.
	Fuente: <i>squat_analyzer_40.py — BioMove v4.0; González-Badillo & Sánchez-Medina (2010). Int. J. Sports Med.</i>

BestRep (Mejor/Peor Repetición) <i>Base de datos / Funcionalidad</i>	Entidad SQLAlchemy que almacena la mejor y peor repetición histórica del usuario por ejercicio (<code>rep_type</code> : "best"/"worst"). Se actualiza automáticamente por la función <code>update_best_rep()</code> en <code>tasks.py</code> de Celery tras cada análisis completado. Permite la comparativa histórica de técnica en <code>BestRepScreen</code> .
	Fuente: <i>best_reps_router.py — BioMove v4.0; tasks.py Celery worker.</i>

BM (clase de tema) <i>Flutter / Diseño</i>	Clase estática que centraliza todos los colores, gradientes y utilidades visuales de la app en <code>app_theme.dart</code> . Sus constantes principales: <code>BM.primary=#6C63FF</code> (morado), <code>BM.accent=#00D4AA</code> (verde turquesa), <code>BM.bg=#07070F</code> (fondo oscuro), <code>BM.card=#161624</code> , <code>BM.textPrimary=#F0F0FF</code> . Todas las pantallas deben usar <code>BM.*</code> (nunca colores hardcodeados).
	Fuente: <i>app_theme.dart — BioMove v4.0 / Flutter Material Design 3.</i>

BioBottomNav <i>Flutter / UI</i>	Widget de navegación inferior personalizado de BioMove. Contiene 5 pestañas: Inicio (<code>/dashboard</code>), Capturar (<code>/capture</code>), Progresión (<code>/progression</code>), Chat (<code>/chat</code>) y Perfil (<code>/profile</code>). Implementado como widget reutilizable en
--	---

	common_widgets.dart. Bug corregido: la pestaña Progresión no tenía context.go("/progression") asignado.
	Fuente: <i>common_widgets.dart</i> — <i>BioMove v4.0</i> .

Butt Wink <i>Biomecánica</i>	Retroversión pélvica al final de la fase excéntrica de la sentadilla. Ocurre cuando la pelvis se bascula hacia atrás (posteroversión) al llegar a la profundidad máxima, aumentando el riesgo lumbar. Calculado como ángulo de inclinación pélvica relativa en vista lateral. Umbral de riesgo: >15° de retroversión.
	Fuente: <i>Schoenfeld, B.J. (2010). Squatting Kinematics and Kinetics. J Strength Cond Res. / squat_analyzer_40.py.</i>

— C —

Celery <i>Backend / Arquitectura</i>	Framework de colas de tareas distribuidas para Python. BioMove usa Celery 5.4 con Redis como broker (puerto 6379) y pool=solo en Windows (prefork causa PermissionError WinError 5). Define 4 colas de prioridad: critical (admin), high (coach), normal (atleta), low (background). Todas las tareas usan bind=True para manejo de excepciones.
	Fuente: <i>Celery 5.4 Documentation (celeryq.dev).</i> / <i>BioMove tasks.py</i> — <i>workers/tasks.py</i> .

Clasificador IA (5 clases) <i>Inteligencia Artificial / ML</i>	Modelo de machine learning implementado con scikit-learn en classifier.py. Clasifica cada repetición de sentadilla en 5 clases técnicas: excelente, buena, aceptable, mejoras, riesgo. Entrenado con 43 features (PARAM_COLS) extraídos del pipeline biomecánico. El archivo .pkl más reciente se carga automáticamente desde backend/models/squat_clf_v*.pkl. Si no existe .pkl, usa fallback a reglas fijas.
	Fuente: <i>classifier.py</i> — <i>BioMove v4.0</i> ; <i>Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.</i>

CRISP-DM <i>Metodología</i>	Cross-Industry Standard Process for Data Mining. Metodología de 6 fases para proyectos de minería de datos e IA: Business Understanding → Data Understanding → Data Preparation → Modeling → Evaluation → Deployment. Aplicada en BioMove para los sprints 5-6 (generación y etiquetado del dataset de sentadillas, entrenamiento del clasificador scikit-learn).
	Fuente: <i>Chapman, P. et al. (1999). CRISP-DM 1.0. SPSS Inc.</i>

CoachAthleteLink	Entidad SQLAlchemy que gestiona la relación de vinculación entre coaches y atletas. Campos: id (UUID PK), coach_id
-------------------------	--

<i>Base de datos / Funcionalidad</i>	(FK→users), athlete_id (FK→users), status (ENUM: pending/active/inactive), linked_at (TIMESTAMP). El atleta inicia la solicitud; el coach la activa. Un coach puede tener múltiples atletas vinculados.
	Fuente: <i>models.py</i> — <i>BioMove v4.0</i> ; <i>coach_router.py</i> .

— D —

Dio <i>Flutter / HTTP</i>	Biblioteca HTTP para Flutter que gestiona todas las comunicaciones con el backend FastAPI. Configurado como singleton (<i>api_service.dart</i>) con: interceptor de Firebase JWT (añade Bearer token a cada request), retry automático en 401 (token renovado), <i>sendTimeout</i> en <i>BaseOptions</i> (no en <i>Options</i> del request individual). El multipart/form-data NO debe llevar Content-Type manual (Dio genera el boundary automáticamente).
	Fuente: <i>Dio 5.x</i> (<i>pub.dev</i>). / <i>api_service.dart</i> — <i>BioMove v4.0</i> .

Dorsiflexión de tobillo <i>Biomecánica</i>	Ángulo de flexión dorsal del tobillo durante la sentadilla, expresado en grados. Indica cuánto el tobillo puede flexionarse hacia adelante sin elevar el talón. Umbral funcional: $\geq 25^\circ$ para profundidad adecuada. Limitaciones en dorsiflexión ($< 25^\circ$) pueden compensarse con elevación de talón y contribuyen al butt wink. Calculado en vista lateral por <i>MediaPipe</i> .
	Fuente: <i>Mauntel et al. (2017). The effects of dorsiflexion range of motion. JOSPT.</i> / <i>squat_analyzer_40.py</i> .

— E —

EMA (Exponential Moving Average) <i>Inteligencia Artificial / Modelo personal</i>	Promedio móvil exponencial con factor de suavizado $\alpha=0.08$. Fórmula: $EMA_t = \alpha \times x_t + (1-\alpha) \times EMA_{t-1}$. Usada en <i>personal_model.py</i> de <i>BioMove</i> para actualizar el modelo IA personal de cada usuario sobre 18 parámetros biomecánicos clave. $\alpha=0.08$ pondera más el historial que los valores recientes, generando un modelo estable.
	Fuente: <i>personal_model.py</i> — <i>BioMove v4.0</i> ; <i>Box, G.E.P. et al. (2015). Time Series Analysis. Wiley.</i>

ENUM (tipo enumerado) <i>Base de datos / SQLAlchemy</i>	Tipo de columna SQLAlchemy que solo acepta un conjunto predefinido de valores string. <i>BioMove</i> usa ENUMs para: role (athlete/coach/admin), video_view (lateral/frontal/both), status (pending/processing/completed/failed), phase (collecting/learning/active/optimizing), entre otros. IMPORTANTE: el valor "auto" no existe en <i>VideoView</i> y causa <i>LookupError</i> (bug crítico corregido en Sprint 16).
---	--

	Fuente: <i>SQLAlchemy 2.x Documentation (sqlalchemy.org).</i> / <i>BioMove models.py.</i>
--	--

— F —

FastAPI <i>Backend / Framework</i>	Framework web moderno de Python (Python 3.10) para construcción de APIs REST de alto rendimiento basado en tipos (Pydantic). BioMove usa FastAPI con Uvicorn (ASGI) en puerto 8000. Características usadas: async/await en todos los endpoints, Pydantic para validación de datos (detecta weight_kg como String causando 422), dependency injection (get_current_user), StaticFiles para servir videos.
	Fuente: <i>FastAPI 0.100+ Documentation (fastapi.tiangolo.com).</i> / <i>BioMove app/main.py.</i>

Firestore Auth <i>Autenticación / Seguridad</i>	Servicio de autenticación de Google Firebase usado en BioMove EXCLUSIVAMENTE para gestión de identidades (JWT). Soporta Google Sign-In y email/contraseña. El token JWT se verifica en cada request del backend con verify_firebase_token() de firebase_admin. IMPORTANTE: Firebase Storage NO se usa en BioMove; los videos se almacenan localmente en FastAPI StaticFiles.
	Fuente: <i>Firebase Auth Documentation (firebase.google.com).</i> / <i>BioMove firebase_service.py.</i>

FeedbackItem <i>Base de datos / Funcionalidad</i>	Entidad SQLAlchemy que almacena los errores técnicos detectados automáticamente en una sesión de análisis. Campos clave: error_type (código del error), severity (low/medium/high), message (descripción en español), correction (corrección recomendada), frequency (número de reps con el error), frequency_pct (porcentaje), is_injury_risk (booleano).
	Fuente: <i>feedback_items (SQLAlchemy) — BioMove v4.0; pipeline.py.</i>

Flower <i>Backend / Monitoreo</i>	Herramienta web de monitoreo para Celery. Disponible en http://localhost:5555 cuando el sistema está en ejecución. Muestra en tiempo real: workers activos, tareas en cola, tareas completadas/fallidas, tiempos de procesamiento. Incluido en start_all.bat de BioMove.
	Fuente: <i>Flower 2.x Documentation (flower.readthedocs.io).</i> / <i>BioMove start_all.bat.</i>

— G —

GlassCard	Widget reutilizable en common_widgets.dart que implementa el efecto glassmorphism (vidrio esmerilado) con
------------------	---

<i>Flutter / UI</i>	BackdropFilter, gradientes de color primario/acento y bordes suaves. Usado en el DashboardScreen, ResultsScreen y componentes principales para el diseño oscuro y moderno de BioMove.
	Fuente: <i>common_widgets.dart</i> — <i>BioMove v4.0; Flutter BackdropFilter</i> .

GoRouter <i>Flutter / Navegación</i>	Biblioteca de navegación declarativa para Flutter (GoRouter 14.x). Gestiona las rutas de BioMove: /login, /register, /onboarding, /dashboard, /capture, /results, /history, /progression, /chat/:userId, /coach, /coach/athlete/:id, /best-reps, etc. CRÍTICO: debe inicializarse en initState() del GoRouterWrapper, NUNCA en build() (causa crash de navegación por reinicialización continua).
	Fuente: <i>GoRouter 14.x (pub.dev/go_router)</i> . / <i>BioMove app_router.dart</i> .

González-Badillo, tabla VBT <i>Biomecánica / VBT</i>	Tabla de referencia empírica que correlaciona velocidad de barra (m/s) con porcentaje del 1RM y RPE (Rating of Perceived Exertion). Implementada en vbt_calculator.py de BioMove. Permite estimar carga de entrenamiento solo con la velocidad de barra, sin necesidad de tests de 1RM frecuentes.
	Fuente: <i>González-Badillo, J.J. & Sánchez-Medina, L. (2010). Movement velocity as a measure of loading intensity. Int. J. Sports Med., 31(5), 347–352.</i>

— H —

Hip_y (coordenada Y de cadera) <i>Biomecánica / Pipeline</i>	Coordenada Y normalizada de los keypoints de cadera en MediaPipe. Fue el primer enfoque de detección de repeticiones en BioMove v1.x, pero fue abandonado porque no funciona correctamente en vista frontal (la cadera no se desplaza verticalmente de forma consistente). Reemplazado por la detección de ángulo de rodilla, que funciona tanto en vista lateral como frontal.
	Fuente: <i>pipeline.py (comentario histórico)</i> — <i>BioMove v4.0</i> .

— I —

IIG (Índice de Interacción Global) <i>Calidad de software / ISO 25010</i>	Métrica compuesta que integra el Índice de Aprendizabilidad (IA) y el Índice de Operabilidad (IO) mediante ponderación diferenciada: $IIG = IA \times 0.40 + IO \times 0.60$. Hipótesis de la tesis: $IIG > 70\%$. Resultado BioMove: $IIG = 84.13\%$ (supera el umbral). La ponderación mayor de IO refleja que la operabilidad cotidiana es más crítica que la curva de aprendizaje inicial.
---	--

	Fuente: ISO/IEC 25010:2023; ISO/IEC 25023:2016. <i>Ficha métrica USQ-INTER-01 — BioMove tesis.</i>
--	---

IA (Índice de Aprendizizabilidad) <i>Calidad de software / ISO 25010</i>	Métrica ISO/IEC 25010:2023 que evalúa qué tan fácilmente un usuario nuevo puede aprender a usar el sistema. Medida con 6 ítems (A1-A6) del cuestionario híbrido en escala Likert 1-7. Fórmula: $IA = (\sum P_{ij} / V_{\max}) \times 100\%$. Resultado BioMove con 10 participantes: IA = 82.86% (umbral objetivo: >70%).
	Fuente: ISO/IEC 25010:2023; ISO/IEC 25023:2016. <i>Ficha métrica USQ-LEARN-01. / BioMove tesis Capítulo IV.</i>

IO (Índice de Operabilidad) <i>Calidad de software / ISO 25010</i>	Métrica ISO/IEC 25010:2023 que evalúa la facilidad de uso del sistema en operación cotidiana. Medida con 9 ítems (O1-O9) del cuestionario en escala Likert 1-7. Fórmula: $IO = (\sum P_{ij} / V_{\max}) \times 100\%$. Resultado BioMove: IO = 85.41% (umbral objetivo: >70%). El ítem O1 (semáforo de validación) obtuvo la media más alta: 6.27.
	Fuente: ISO/IEC 25010:2023; ISO/IEC 25023:2016. <i>Ficha métrica USQ-OPER-01. / BioMove tesis Capítulo IV.</i>

ISO/IEC 25010:2023 <i>Norma de calidad</i>	Norma internacional de calidad de producto software. Define 8 características: Adecuación funcional, Eficiencia de desempeño, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad y Portabilidad. BioMove evalúa dos subcaracterísticas de Usabilidad: Aprendizizabilidad (learnable) y Operabilidad (operable).
	Fuente: ISO (2023). <i>ISO/IEC 25010:2023 — Systems and software Quality Requirements and Evaluation (SQuaRE).</i> Geneva.

— J —

JWT (JSON Web Token) <i>Seguridad / Autenticación</i>	Estándar abierto (RFC 7519) para transmitir información de autenticación como objeto JSON firmado digitalmente. Firebase Auth genera JWTs que BioMove usa en cada request al backend (header: Authorization: Bearer {token}). El backend verifica el JWT con <code>firebase_admin.auth.verify_id_token()</code> . El frontend ejecuta <code>_waitForToken()</code> (loop 10×500ms) para asegurar disponibilidad del token antes del primer request.
	Fuente: RFC 7519 (JWT). <i>Firebase Auth Documentation.</i> / <i>BioMove api_service.dart + firebase_service.py.</i>

job_id <i>Backend / Celery</i>	UUID único generado para cada trabajo de análisis de video. Retornado por POST /videos/upload-and-analyze. El frontend usa el job_id para: polling de estado (GET /videos/{job_id}/status cada 3s), obtención de resultados
--	---

	(GET /videos/{job_id}/results). Mapea a la clave primaria de la tabla video_jobs.
	Fuente: <i>videos_router.py</i> — BioMove v4.0; <i>video_jobs</i> (SQLAlchemy).

— K —

Keypoints (puntos articulares) <i>Visión artificial / MediaPipe</i>	33 coordenadas espaciales (x, y, z, visibility) detectadas por MediaPipe Pose en cada frame de video. Representan articulaciones y puntos anatómicos del cuerpo humano: nariz, ojos, orejas, hombros, codos, muñecas, caderas, rodillas, tobillos, pies y punta de pies. Base del pipeline de análisis biomecánico de BioMove.
	Fuente: Google (2023). <i>MediaPipe Pose Landmarker. AI for Developers</i> (ai.google.dev).

Knee_angle_min (ángulo mínimo de rodilla) <i>Biomecánica</i>	Parámetro biomecánico principal calculado por <i>squat_analyzer_40.py</i> . Representa el ángulo mínimo alcanzado en la articulación de la rodilla durante la fase bottom de la sentadilla. Es también el indicador de detección de repeticiones (umbral: $\leq 95^\circ$ para profundidad adecuada, equivalente a sentadilla paralela). Funciona en vista lateral y frontal (a diferencia de hip_y).
	Fuente: <i>squat_analyzer_40.py</i> — BioMove v4.0; NSCA (2016). <i>Essentials of Strength Training and Conditioning</i> .

— M —

MAE (Error Absoluto Medio) <i>Evaluación IA / Biomecánica</i>	Mean Absolute Error. Métrica de evaluación del pipeline biomecánico que mide la diferencia promedio en grados entre los ángulos calculados por MediaPipe+squat_analyzer_40.py y los valores de referencia validados. Resultado BioMove: MAE = 14.8° (por debajo del umbral objetivo de $\leq 15^\circ$). Calculado con n=10 participantes, múltiples repeticiones.
	Fuente: BioMove tesis — Capítulo IV / <i>squat_analyzer_40.py</i> ; Hastie, T. et al. (2009). <i>The Elements of Statistical Learning</i> .

MediaPipe Pose <i>Visión artificial / IA</i>	Framework de Google para detección y seguimiento de puntos articulares humanos en tiempo real. BioMove usa model_complexity=2 (máxima precisión de los 3 niveles disponibles, máxima latencia). Detecta 33 keypoints por frame en videos de sentadilla. Se ejecuta en el servidor backend Python (no en el dispositivo móvil). La orientación del video (lateral/frontal) se auto-detecta analizando el ancho de hombros.
	Fuente: Google LLC (2023). <i>MediaPipe Pose</i> (ai.google.dev). / <i>pipeline.py</i> — BioMove v4.0.

model_complexity=2 <i>Visión artificial / MediaPipe</i>	Parámetro de configuración de MediaPipe Pose. El valor 2 activa el modelo más complejo y preciso (de 0 a 2), maximizando la exactitud de detección de keypoints. Elegido en BioMove para alcanzar el MAE objetivo $\leq 15^\circ$. Implica mayor tiempo de procesamiento por frame, compensado por el skip de frames pares y el procesamiento asíncrono con Celery.
	Fuente: <i>pipeline.py</i> — <i>BioMove v4.0</i> ; <i>MediaPipe Pose Documentation</i> (Google, 2023).

MVVM (Model-View-ViewModel) <i>Arquitectura / Flutter</i>	Patrón arquitectónico implementado en el frontend Flutter de BioMove. Flujo estricto: View (pantallas Flutter) → ViewModel (ChangeNotifier con Provider) → Repository (abstracción de datos) → Service (Dio, Firebase, WebSocket). La View NUNCA llama directamente al Repository. Facilita testabilidad, modularidad y cumplimiento de ISO 25010 (Mantenibilidad).
	Fuente: <i>BioMove app/viewmodel/</i> — <i>Flutter MVVM.</i> / <i>Microsoft</i> (2012). <i>The MVVM Pattern</i> .

— O —

Onboarding <i>Flutter / UX</i>	Proceso inicial de configuración de perfil del usuario al registrarse por primera vez. OnboardingScreen captura: height_cm, weight_kg, age, sex, training_years. El campo onboarding_done se guarda en SQLite (tabla users) y se cachea localmente en SharedPreferences (key "onboarding_done_local") para resiliencia ante fallo de red. GoRouter redirige: authenticated+!onboarding_done → /onboarding.
	Fuente: <i>onboarding_viewmodel.dart</i> — <i>BioMove v4.0</i> ; <i>auth_router GoRouter redirect</i> .

one_rm (1RM — Una Repetición Máxima) <i>Biomecánica / VBT</i>	Peso máximo que un deportista puede levantar en una sola repetición de un ejercicio. En BioMove se calcula con dos fórmulas clásicas en strength_calculator.py: Epley: $1RM = weight \times (1 + reps/30)$; Brzycki: $1RM = weight / (1.0278 - 0.0278 \times reps)$. También se puede estimar en tiempo real mediante VBT: predict_1rm_from_velocity(weight, velocity_ms).
	Fuente: <i>Epley, B. (1985). Poundage Chart. Boyd Epley Workout.</i> / <i>Brzycki, M. (1993). Strength Testing. JOHPERD.</i>

— P —

Pipeline biomecánico <i>Backend / Análisis</i>	Secuencia de procesamiento de video implementada en pipeline.py + squat_analyzer_40.py + classifier.py de BioMove. Flujo: OpenCV (frames impares) → MediaPipe (33 keypoints) → máquina de estados (ángulo rodilla) → squat_analyzer_40.py (40+ params) → classifier.py (5 clases) → personal_model.py (EMA $\alpha=0.08$) → video_annotator.py (video anotado). Ejecutado de forma asíncrona por Celery worker.
	Fuente: <i>pipeline.py — BioMove v4.0; tasks.py Celery worker.</i>

progress_pct (porcentaje de progreso) <i>Backend / UX</i>	Campo INTEGER (0-100) en la tabla video_jobs. Actualizado por el worker Celery cada 5% del análisis. El frontend Flutter hace polling GET /videos/{job_id}/status cada 3 segundos y muestra una ProgressBar con este valor. La escala: 0-20% (descarga/preparación), 20-60% (detección MediaPipe), 60-80% (análisis 40 params), 80-95% (clasificación + EMA), 95-100% (video anotado).
	Fuente: <i>tasks.py Celery worker — BioMove v4.0; video_jobs (SQLAlchemy).</i>

Provider <i>Flutter / Estado</i>	Paquete de gestión de estado reactivo para Flutter. BioMove lo usa con ChangeNotifier y Consumer/context.watch<T>() para la comunicación reactiva entre ViewModels y Views. Configurado con MultiProvider en main.dart. Cada dominio tiene su propio ViewModel: AuthViewModel, AnalysisViewModel, ProgressionViewModel, ChatViewModel, BestRepViewModel.
	Fuente: <i>Provider 6.x (pub.dev). / BioMove main.dart + viewmodel/.</i>

— R —

Redis <i>Backend / Infraestructura</i>	Base de datos en memoria de alta velocidad usada como broker de mensajería para Celery en BioMove. Puerto: 6379. En desarrollo local, ejecutado mediante Docker Desktop. Redis recibe las tareas Celery y las distribuye a los workers disponibles según la cola asignada (critical/high/normal/low). Sin Redis, Celery no puede funcionar.
	Fuente: <i>Redis 7.x Documentation (redis.io). / BioMove .env: REDIS_URL=redis://localhost:6379.</i>

Repository Pattern <i>Arquitectura / Flutter</i>	Patrón de diseño que desacopla la lógica de acceso a datos de la lógica de negocio (ViewModels). BioMove tiene 4 repositorios: analysis_repository.dart, user_repository.dart, progression_repository.dart, chat_repository.dart. Los repositorios abstraen si los datos vienen de la API REST, de
--	--

	caché local (SharedPreferences) o de Firebase. Los ViewModels no conocen la fuente de datos.
	Fuente: <i>Fowler, M. (2003). Patterns of Enterprise Application Architecture. Addison-Wesley. / BioMove repository/.</i>

RPE (Rating of Perceived Exertion) <i>Ciencias del deporte / VBT</i>	Escala de percepción subjetiva del esfuerzo. En el contexto VBT de BioMove, se estima automáticamente a partir de la velocidad de barra mediante <code>velocity_to_rpe(vel)</code>, sin necesidad de auto-reporte del atleta. Escala 1-10 donde 10=máximo esfuerzo. $RPE \geq 9$ con pérdida de velocidad $>20\%$ activa <code>should_stop:True</code>. Fuente: <i>Borg, G. (1982). Psychophysical bases of perceived exertion. Med. Sci. Sports Exerc. / vbt_calculator.py.</i>
--	--

— S —

ScoreGauge <i>Flutter / UI</i>	Widget semicircular personalizado en <code>common_widgets.dart</code> que visualiza el score de técnica (0-100) del análisis. Incluye animación de aparición y efecto de glow pulsante. Colores: verde (≥ 70, excelente), amarillo (50-69, regular), rojo (< 50, mejorar). Implementado con <code>CustomPainter</code> y animaciones <code>flutter_animate</code>. Fuente: <i>common_widgets.dart — BioMove v4.0; flutter_animate package.</i>
--	---

ScRUM <i>Metodología</i>	Marco de trabajo ágil para desarrollo de software basado en sprints iterativos, roles definidos (Product Owner, Scrum Master, Development Team) y artefactos (Product Backlog, Sprint Backlog, Increment). BioMove usó SCRUM combinado con CRISP-DM: 16 sprints de 2 semanas (enero-junio 2026), con 25 ítems en el Product Backlog (PB1-PB25), todos completados. Fuente: <i>Schwaber, K. & Sutherland, J. (2020). The Scrum Guide. Scrum.org.</i>
------------------------------------	--

SharedPreferences <i>Flutter / Almacenamiento local</i>	Mecanismo de almacenamiento local clave-valor en Flutter para datos pequeños. Usos en BioMove: (1) "onboarding_done_local": caché del estado de onboarding para resiliencia offline ante fallo de red al cargar perfil; (2) configuración de cámara (resolución, orientación preferida). Es la única fuente de datos local del frontend; toda la información principal está en el backend SQLite. Fuente: <i>shared_preferences 2.x (pub.dev). / BioMove auth_viewmodel.dart + camera_service.dart.</i>
---	--

Shapiro-Wilk (W) <i>Estadística / Metodología</i>	Prueba de normalidad estadística. BioMove la aplica para determinar la distribución de los índices IA, IO e IIG antes de seleccionar la prueba de contraste de hipótesis. Resultado: $W=0.821$, $p=0.042$ ($p<0.05 \rightarrow$ distribución no normal $\rightarrow$ se usa Wilcoxon). La prueba Shapiro-Wilk es más potente para muestras pequeñas ($n=10$).
	Fuente: <i>Shapiro, S.S. & Wilk, M.B. (1965). An analysis of variance test for normality. Biometrika, 52(3/4). / BioMove tesis Capítulo IV.</i>

SQLAlchemy async <i>Backend / ORM</i>	ORM (Object-Relational Mapper) de Python en versión asíncrona (2.x). BioMove usa AsyncSession, create_async_engine, aiosqlite (dev) y asyncpg (prod). Mapea las 12 clases Python (User, VideoJob, TrainingSession, etc.) a tablas SQL. Las migraciones se hacen manuales (sin Alembic) en desarrollo. create_tables() crea todas las tablas al iniciar FastAPI.
	Fuente: <i>SQLAlchemy 2.x Documentation (sqlalchemy.org). / BioMove database.py.</i>

— T —

Técnica de sentadilla (clasificación) <i>Biomecánica / IA</i>	Sistema de 5 niveles para clasificar la calidad técnica de una repetición de sentadilla en BioMove: Excelente (score ≥ 85 , todos los parámetros en rango óptimo), Buena (70-84, pequeñas desviaciones), Aceptable (55-69, mejoras recomendadas), Mejoras (40-54, errores significativos), Riesgo (<40 , errores con riesgo de lesión). Asignado por el clasificador scikit-learn o fallback a reglas fijas.
	Fuente: <i>classifier.py — BioMove v4.0; squat_analyzer_40.py.</i>

TrainingSession <i>Base de datos</i>	Entidad central del sistema que registra los resultados de cada sesión de análisis de sentadilla completada. Una sesión es el resultado de un VideoJob exitoso (status="completed"). Contiene: technique_score, total_reps, fatigue_detected, video_view, shared_with_coach. Relacionada con múltiples Repetitions y FeedbackItems.
	Fuente: <i>training_sessions (SQLAlchemy) — BioMove v4.0 / models.py.</i>

— U —

UserAIModel <i>Base de datos / IA personal</i>	Entidad SQLAlchemy con relación 1:1 con users que almacena el modelo IA personal del atleta. Campos: phase (fase actual), total_reps (acumuladas), params_model (JSON con valores EMA de 18 parámetros), morphology (JSON con morfología detectada), improvement_trends (JSON con
--	---

	tendencias). Se actualiza automáticamente en Celery post-análisis.
	Fuente: <i>user_ai_models (SQLAlchemy) — BioMove v4.0; personal_model.py.</i>

Uvicorn <i>Backend / Servidor</i>	Servidor web ASGI (Asynchronous Server Gateway Interface) de alto rendimiento para Python. Ejecuta FastAPI en BioMove en el puerto 8000. Iniciado con: <code>uvicorn app.main:app --host 0.0.0.0 --port 8000</code> . En desarrollo, se inicia mediante <code>start_all.bat</code> como un proceso separado en su propia ventana de terminal.
	Fuente: <i>Uvicorn Documentation (uvicorn.org). / BioMove start.py + start_all.bat.</i>

— V —

VBT (Velocity Based Training) <i>Ciencias del deporte / Biomecánica</i>	Metodología de entrenamiento de fuerza que usa la velocidad de ejecución como indicador de la intensidad del entrenamiento (en lugar de solo el % del 1RM). BioMove implementa VBT en <code>vbt_calculator.py</code> : <code>velocity_to_rpe(vel)</code> calcula RPE y %1RM; <code>predict_1rm_from_velocity(weight, vel)</code> estima el 1RM del día. Pérdida >20% de velocidad en un set activa <code>should_stop=True</code> .
	Fuente: <i>González-Badillo & Sánchez-Medina (2010). Int. J. Sports Med. / vbt_calculator.py — BioMove v4.0.</i>

Valgo de rodilla (knee valgus) <i>Biomecánica</i>	Desviación medial de la rodilla durante la sentadilla. Se manifiesta como el "colapso" de las rodillas hacia adentro. En BioMove se mide como <code>left_valgus</code> y <code>right_valgus</code> (en grados) en vista frontal. Umbral seguro: <10°. Está asociado con mayor riesgo de lesión del LCA (<code>acl_risk_score</code>). Uno de los errores más frecuentes detectados por el sistema.
	Fuente: <i>Hewett, T.E. et al. (2005). Biomechanical Measures of Neuromuscular Control. Am. J. Sports Med. / squat_analyzer_40.py.</i>

video_annotator.py <i>Backend / Pipeline</i>	Módulo Python en BioMove que genera el video anotado a partir del video original y los resultados del análisis. Añade: overlay del esqueleto RGB (keypoints y conexiones coloreadas por riesgo), scores por repetición, alertas de postura, barra de progreso temporal. Los frames pares (saltados en el análisis) se rellenan con <code>cv2.addWeighted(frame,0.45,last_annotated,0.55,0)</code> . El video se recorta desde <code>first_rep_start</code> hasta <code>last_rep_end</code> .
	Fuente: <i>video_annotator.py — BioMove v4.0; OpenCV (cv2) Documentation.</i>

VideoJob <i>Base de datos / Backend</i>	Entidad SQLAlchemy que rastrea el ciclo de vida de cada trabajo de análisis de video. Estados: pending → processing → completed failed. El worker Celery actualiza progress_pct (0-100%) durante el procesamiento. Al completarse almacena annotated_download_url. El frontend hace polling de GET /videos/{job_id}/status cada 3 segundos.
	Fuente: <i>video_jobs (SQLAlchemy) — BioMove v4.0; videos_router.py.</i>

— W —

WebSocket <i>Backend / Flutter / Chat</i>	Protocolo de comunicación bidireccional en tiempo real sobre TCP. BioMove implementa WebSocket en /chat/ws/{user_id} (FastAPI + WebSocketChannel en Flutter). Incluye ping keepalive cada 30 segundos para mantener la conexión activa. Soporta 3 tipos de mensajes: text (texto libre), analysis_ref (referencia a sesión) y video_ref (referencia a video).
	Fuente: <i>WebSocket RFC 6455. / chat_router.py — BioMove v4.0; websocket_service.dart.</i>

Wilcoxon (prueba de Rangos con Signo) <i>Estadística / Metodología</i>	Prueba estadística no paramétrica usada para contrastar hipótesis cuando los datos no siguen distribución normal. BioMove la aplica para verificar si los índices IA, IO e IIG superan estadísticamente el umbral H_0 : IIG $\leq$ 70%. Resultado: $Z=-2.803$, $p=0.002$ ($p<0.05 \rightarrow$ se rechaza H_0 , IIG es estadísticamente $>70\%$). Tamaño del efecto $r=0.80$ (grande según Cohen).
	Fuente: <i>Wilcoxon, F. (1945). Individual comparisons by ranking methods. Biometrics Bulletin. / BioMove tesis Capítulo IV.</i>

— A —

_waitForToken() <i>Flutter / Autenticación</i>	Función interna de AuthViewModel en BioMove. Ejecuta un loop de hasta 10 iteraciones con pausas de 500ms cada una, esperando a que Firebase Auth tenga el token JWT disponible. Previene errores 401 que ocurrían cuando el frontend hacía requests al backend antes de que Firebase inicializara completamente. Llamada obligatoria antes de cualquier request POST al backend al iniciar sesión.
	Fuente: <i>auth_viewmodel.dart — BioMove v4.0.</i>